

Aula 1 — Arquitetura do PIC18

Memória, ciclo de instrução, árvore de clock e ambiente de desenvolvimento

Microcontroladores — DENE/UFMT

Objetivos da aula

- Descrever a organização interna do PIC18F4550: núcleo, memórias e barramentos, e relacionar cada espaço de memória a construções da linguagem C.
- Calcular o tempo de execução a partir da frequência do oscilador, compreendendo a relação entre ciclo de oscilador e ciclo de instrução.
- Interpretar o mapa de memória de dados: bancos, registrador de seleção e banco de acesso.
- Configurar a árvore de clock e as palavras de configuração, identificando os bits que mais causam falhas silenciosas.
- Percorrer a cadeia de ferramentas do código-fonte ao chip gravado, sabendo onde ler o consumo de memória do projeto.

1 O que há dentro do encapsulamento

O encontro anterior definiu o microcontrolador pela integração. Vale agora abrir o chip e nomear o que está integrado.

Recurso	PIC18F4550
Núcleo	8 bits, arquitetura Harvard, multiplicador por hardware
Memória de programa	32 KB de Flash, equivalentes a 16 384 instruções de uma palavra
Memória de dados	2 048 bytes de RAM estática
Memória não volátil de dados	256 bytes de EEPROM
Frequência máxima	48 MHz, com 12 milhões de instruções por segundo
Portas de entrada e saída	35 pinos no encapsulamento de 40 vias
Conversor analógico-digital	10 bits, 13 canais
Comparadores	Dois, com referência programável
Temporizadores	Quatro, sendo um de 8 bits e três de 16
Comunicação	Serial assíncrona, síncrona mestre-escravo e controlador USB

Tabela 1: Recursos do dispositivo usado na disciplina.

Duas linhas dessa tabela merecem comentário imediato. O **multiplicador por hardware** executa uma multiplicação de 8 por 8 bits em um único ciclo — na família anterior, a mesma operação exigia uma rotina de dezenas de ciclos, o que tornava proibitiva qualquer aritmética não trivial. E o **controlador USB**, que não será usado no projeto, explica várias peculiaridades da configuração de clock que aparecem adiante e costumam confundir quem só quer piscar um LED.

2 Harvard e o ciclo de instrução

Definição --- arquitetura Harvard

Organização em que a memória de programa e a de dados ocupam espaços de endereçamento distintos, com barramentos próprios, podendo inclusive ter larguras diferentes. Contrapõe-se à arquitetura de Von Neumann, em que ambas compartilham um único espaço e um único barramento.

No PIC18F4550, o barramento de programa tem 16 bits de largura — o tamanho de uma instrução — e o de dados tem 8. Isso permite buscar a próxima instrução **enquanto** a atual é executada, já que as duas operações usam barramentos diferentes e não competem entre si.

Observação

A consequência prática é o **paralelismo de dois estágios**: busca e execução ocorrem simultaneamente, e o processador completa uma instrução por ciclo de instrução. As exceções são os desvios, que descartam a instrução já buscada e consomem dois ciclos — um detalhe que importa quando se conta ciclos para temporização precisa.

2.1 A relação entre oscilador e instrução

Derivação

Cada ciclo de instrução consome **quatro** ciclos do oscilador:

$$T_{cy} = \frac{4}{f_{osc}}$$

Com o oscilador em 48 MHz:

$$T_{cy} = \frac{4}{48 \cdot 10^6} \approx 83,3 \text{ ns}$$

o que corresponde a 12 milhões de instruções por segundo. Um trecho de código com 120 instruções, sem desvios, executa em cerca de 10 µs.

Atenção

Confundir frequência do oscilador com frequência de instrução é o erro mais comum nos cálculos de temporização do semestre — e ele reaparece nos encontros de temporizadores e de modulação, porque todos os divisores de frequência partem de T_{cy} , não de f_{osc} . Um erro de fator quatro num cálculo de período é quase sempre este.

3 A memória de programa

O contador de programa tem 21 bits, o que permite endereçar 2 MB. O dispositivo implementa 32 KB — o restante do espaço existe na arquitetura, mas não no silício. Três endereços dentro dessa faixa têm significado fixo:

Endereço	Papel	Comentário
0x0000	Vetor de reinicialização	Onde a execução começa após qualquer reset

Endereço	Papel	Comentário
0x0008	Vetor de interrupção de alta prioridade	Detalhado na Aula 9
0x0018	Vetor de interrupção de baixa prioridade	Idem

Tabela 2: Endereços fixos da memória de programa.

Observação

O espaço entre 0x0008 e 0x0018 tem apenas 16 bytes — oito instruções. Não cabe um tratador de interrupção ali. O que o compilador coloca nesse endereço é um desvio para o tratador propriamente dito, alocado em outro ponto. Saber disso evita a surpresa ao inspecionar a listagem de código.

A memória de programa também pode ser **lida como dados**, por instruções específicas de leitura de tabela. É o que permite armazenar constantes grandes — mensagens de texto, tabelas de conversão — na Flash, sem consumir os escassos 2 KB de RAM.

3.1 A pilha de hardware

Chamadas de função e interrupções empilham o endereço de retorno numa pilha dedicada, com 31 níveis. Ela é separada da memória de dados: consumir a pilha não consome RAM.

Atenção

Estourar 31 níveis é difícil em código bem estruturado, mas não impossível com recursão ou cadeias muito profundas de chamadas. Diferentemente da família anterior, o PIC18 permite **detectar** o estouro: há um bit de configuração que transforma o estouro em uma reinicialização, e o ponteiro de pilha é legível por software. Um reset inexplicável em código com recursão tem aí a sua primeira hipótese.

4 A memória de dados

Este é o ponto em que a arquitetura de 8 bits mostra sua idade, e também onde o PIC18 se distancia mais claramente do PIC16.

Os 2 048 bytes são organizados em **bancos** de 256 bytes. Como as instruções não têm bits suficientes para endereçar 2 048 posições diretamente, o endereço completo é formado pela concatenação do banco selecionado no registrador BSR com o deslocamento contido na instrução.

Atenção

No PIC18F4550, metade da memória de dados é fisicamente compartilhada com o controlador USB. Com o módulo USB desabilitado — como no projeto desta disciplina — essa região fica disponível como memória de uso geral. É uma peculiaridade deste dispositivo, não da família.

4.1 O banco de acesso

Trocar o banco a cada acesso é caro em ciclos e é a origem da fama de “desconfortável” da família anterior. O PIC18 resolve o problema com uma janela especial:

Definição --- banco de acesso

Região virtual formada pelos 96 primeiros bytes da memória de uso geral reunidos aos registradores de função especial da parte alta do mapa. Toda instrução possui um bit que seleciona essa janela, permitindo acesso **sem alterar** o registrador de banco.

Observação

A consequência é direta e importante: as variáveis mais usadas e todos os registradores de periféricos ficam acessíveis em uma instrução, sem troca de banco. Combinado aos três ponteiros de endereçamento indireto com incremento automático e ao multiplicador por hardware, isso é o que torna o PIC18 uma arquitetura **projetada para receber código de compilador C**, e não apenas tolerante a ele. O PIC16 aceitava C a contragosto; o PIC18 foi desenhado com ele em mente.

4.2 O que o compilador coloca em cada lugar

Construção em C	Onde reside	Observação
Código das funções	Flash	Consome memória de programa
const e literais de texto	Flash	Acesso por leitura de tabela
Variáveis globais e estáticas	RAM	Existem durante toda a execução
Variáveis locais	RAM	Endereços fixos, atribuídos pelo compilador
Variáveis de tratador	RAM, com volatile	Vide Aula 9
Dados persistentes	EEPROM	Escrita explícita; vide Aula 10

Tabela 3: Correspondência entre construções em C e espaços de memória.

Atenção

A quarta linha é contraintuitiva para quem vem de programação em computador. **As variáveis locais não vivem em uma pilha:** o compilador atribui a elas endereços fixos, reaproveitando as mesmas posições entre funções que não coexistem. Daí decorre a não reentrância discutida na Aula 9 — a mesma função chamada do tratador e do laço principal usará as mesmas posições de memória nas duas invocações.

5 A árvore de clock e as palavras de configuração

Aqui está a parte que mais consome tempo de laboratório na primeira sessão, e quase nunca por dificuldade conceitual: por descuido de configuração.

O PIC18F4550 tem uma árvore de clock incomumente complicada para um dispositivo de 8 bits, e a razão é o USB. O controlador exige exatamente 48 MHz, obtidos de um multiplicador de frequência interno que precisa receber, **na sua entrada**, exatamente 4 MHz. Daí a cadeia:

1. o cristal externo entra no dispositivo;
2. um primeiro divisor, configurado em PLLDIV, reduz essa frequência para os 4 MHz exigidos pelo multiplicador;
3. o multiplicador gera a frequência alta interna;

4. um segundo divisor, configurado em CPUDIV, define a partir dela a frequência entregue ao processador.

Exemplo

Com um cristal de 20 MHz, PLLDIV deve dividir por 5, entregando os 4 MHz necessários. Se o cristal for de 8 MHz, o divisor deve ser 2; se for de 4 MHz, o divisor é 1. **O valor correto depende do cristal soldado na placa** — e um valor errado aqui não impede a compilação nem a gravação: o programa simplesmente executa na frequência errada, e todos os tempos ficam proporcionalmente incorretos.

Atenção

Confirme o cristal do kit antes de fixar PLLDIV. O sintoma de um valor errado é característico e enganoso: o LED pisca, mas em ritmo diferente do calculado, e a comunicação serial produz caracteres corrompidos — porque a taxa de símbolos deriva da frequência do processador. A configuração utilizada no laboratório está congelada no Roteiro 0.

5.1 Os bits que causam falhas silenciosas

As palavras de configuração são gravadas junto com o programa e definem o comportamento do dispositivo **antes** da primeira instrução executar. Nenhum erro aqui é detectado pelo compilador.

Bit	Por que ele aparece nesta lista
XINST	Habilita o conjunto estendido de instruções, não suportado pelo compilador usado . Ligado por engano, o programa executa lixo. Deve permanecer desabilitado
LVP	Programação em baixa tensão. Habilitada, mantém um pino da porta B reservado, que deixa de funcionar como entrada e saída comum — e um ruído nesse pino pode colocar o dispositivo em modo de programação
WDT	Cão de guarda. Habilitado sem que o programa o alimente, provoca reinicializações periódicas que se manifestam como “o programa reinicia sozinho”
PBADEN	Define se pinos da porta B iniciam como analógicos ou digitais. O padrão não é digital: um botão ligado a esses pinos parece não funcionar até que isso seja corrigido
MCLRE	Define se o pino de reinicialização externa é reset ou entrada digital. Alterá-lo sem necessidade pode impedir a gravação
FOSC, PLLDIV, CPUDIV	Definem a frequência efetiva. Errados, corrompem toda a temporização e a comunicação serial

Tabela 4: Bits de configuração responsáveis pelas falhas mais frequentes.

Código

```
/* Trecho do bloco de configuracao. Os valores completos
   estao congelados no Roteiro 0 do laboratorio. */

#pragma config PLLDIV    = 5      /* conferir o cristal da placa */
```

```
#pragma config CPUDIV    = OSC1_PLL2
#pragma config FOSC       = HSPLL_HS
#pragma config WDT        = OFF      /* ligado somente na Aula 10 */
#pragma config LVP        = OFF      /* libera o pino da porta B */
#pragma config XINST      = OFF      /* obrigatorio com XC8 */
#pragma config PBDEN      = OFF      /* porta B digital no reset */
#pragma config MCLRE      = ON
```

Observação

Vale registrar o método por trás desta seção. Nenhum desses bits é conceitualmente difícil; todos produzem sintomas que **parecem** defeitos de programa. Quando um código correto se comporta de maneira inexplicável, a palavra de configuração deve ser verificada **antes** de o código ser reescrito.

6 Do código-fonte ao chip

Etapa	Ferramenta	Produto
1	Pré-processador	Código com diretivas resolvidas
2	Compilador	Código de máquina relocável, por módulo
3	Ligador	Programa único, com endereços resolvidos
4	Gerador de imagem	Arquivo .hex, com o conteúdo da Flash
5	Gravador	Dispositivo programado

Tabela 5: Cadeia de ferramentas.

Dois arquivos produzidos nesse caminho são instrumentos de trabalho e não subprodutos descartáveis:

A listagem mostra, lado a lado, cada linha de C e as instruções de máquina geradas. É onde se descobre que uma multiplicação virou uma instrução e uma divisão virou uma chamada de biblioteca de dezenas de ciclos — informação decisiva quando o tempo importa.

O mapa de memória informa quanto de Flash e de RAM o projeto consome. Como o laboratório acrescenta uma camada por semana ao mesmo código, acompanhar esses dois números ao longo do semestre transforma um conceito abstrato em uma curva observável.

Atenção

A memória de programa raramente é o limite deste projeto; os 2 KB de RAM são. Buffers de comunicação, vetores de média móvel e cadeias de texto consomem RAM rapidamente. O estouro é reportado pelo ligador — e é um erro de compilação, não uma falha em execução, o que é uma boa notícia.

7 Transposição

Quem já viu a família PIC16 reconhecerá quase tudo, com quatro diferenças que concentram o essencial: o vetor único de interrupção deu lugar a dois, com prioridade; a pilha passou de 8 para 31 níveis e tornou-se observável; o banco de acesso removeu a troca constante de banco; e surgiram o multiplicador por hardware e os modos de endereçamento indireto que viabilizam C eficiente.

Na direção oposta, rumo às arquiteturas de 32 bits, dois pontos desta aula sobrevivem intactos e um desaparece. Sobrevivem a **árvore de clock** — ainda mais elaborada, com múltiplos multiplicadores e divisores por periférico, e igualmente capaz de invalidar toda a temporização quando mal configurada — e o **orçamento de memória**, que continua sendo lido no mapa gerado pelo ligador. Desaparece o **banqueamento**: com endereçamento linear de 32 bits, bancos e janelas de acesso deixam de existir, e com eles toda uma classe de erros.

8 Exercícios

Exercício 1.1

Um projeto usa cristal de 8 MHz e precisa que o processador opere a 24 MHz. Determine os divisores de entrada e de saída necessários, justificando a restrição que fixa a frequência na entrada do multiplicador. Em seguida, calcule T_{cy} e o tempo de execução de uma rotina de 400 instruções sem desvios.

Exercício 1.2

Explique por que a região entre os dois vetores de interrupção não comporta um tratador e descreva o que o compilador efetivamente coloca nesses endereços.

Exercício 1.3

Um estudante relata que o botão ligado a um pino da porta B não é detectado, embora o mesmo código funcione com o botão movido para a porta D. O programa está correto. Indique a causa mais provável, o bit envolvido e a correção.

Exercício 1.4

Considere um vetor de 64 amostras de 16 bits para média móvel, mais dois buffers de comunicação de 32 bytes cada e uma cadeia de texto constante de 200 caracteres. Calcule o consumo de RAM e o de Flash, indicando em qual memória cada item reside e que fração dos recursos disponíveis é consumida.

Exercício 1.5

A afirmação a seguir é frequentemente repetida: “a arquitetura Harvard é mais rápida que a de Von Neumann”. Discuta em que sentido ela é correta neste dispositivo, apontando o mecanismo concreto responsável pelo ganho, e em que sentido ela é uma simplificação indevida quando se comparam processadores de gerações diferentes.